

UNIDAD DE PROGRAMACIÓN I.

Caracterización de sistemas de IA.

Autor: Pedro Antonio Santiago Santiago.
Fecha: Septiembre 2025

 CC BY-NC-SA 4.0

Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to distribute, remix, adapt, and build upon the material in any medium or format, for noncommercial purposes only. If others modify or adapt the material, they must license the modified material under identical terms.

 **BY:** Credit must be given to you, the creator.

 **NC:**
Only noncommercial use of your work is permitted. *Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.*

 **SA:** Adaptations must be shared under the same terms.

[See the License Deed](#)

Índice.

1. Ficha pedagógica.....	3
2. Introducción.....	4
2.1. Definición de Inteligencia Artificial.....	4
2.2. Evolución histórica.....	5
2.3. Test de Turing.....	6
3. Fundamentos de los sistemas inteligentes.....	8
4. Campos de aplicación.....	10
5. Técnicas de la Inteligencia Artificial.....	12
5.1. Resolución de problemas.....	12
5.1.1. Búsquedas no informadas.....	13
5.1.2. Búsqueda informada.....	18
5.2. Representación e inferencia.....	19
5.2.1. Representación del conocimiento.....	20
5.2.2. Inferencia.....	24
5.3. Procesamiento del lenguaje natural.....	26
5.3.1. Sistemas expertos.....	28
5.4. Robótica.....	29
6. Nuevas formas de interacción.....	34
7. Actividades de autoevaluación.....	36
8. Actividades propuestas.....	36

1. Ficha pedagógica.

Temporalización: 1ª a 2ª semana de octubre. 6 horas.	
OBJETIVOS DIDÁCTICOS	
RESULTADOS DE APRENDIZAJE	RA1
OBJETIVOS GENERALES	a, e, j, k, m, ñ
COMPETENCIAS	a, d, e, k, n, o, p, q
CONTENIDOS	
Fundamentos de los sistemas inteligentes. Campos de aplicaciones. Técnicas de la Inteligencia Artificial. Nuevas formas de interacción.	
CRITERIO DE EVALUACIÓN	a) Se han identificado los principios fundamentales de los sistemas inteligentes. b) Se ha recopilado información sobre campos donde se aplica Inteligencia Artificial. c) Se han identificado las técnicas básicas a utilizar en el entorno de la IA. d) Se han identificado nuevas formas de interacciones en los negocios que mejore la eficiencia operativa.

2. Introducción.

Actualmente las palabras “inteligencia artificial” o IA se escuchan a todas horas, cualquier aplicación/dispositivo/electrodoméstico... utiliza IA, aunque no se sepa ni tan siquiera la definición de la por parte de la mayoría de la gente no conozca ni la evolución de esta, ni los diferentes tipos de IA que existen.

Muchas de las técnicas de IA existen desde hace decenas de años, aunque la potencia de cálculo de los años en que se definieron no hacían viable su aplicación, aunque es en los últimos años, cuando la potencia de cálculo ha permitido aplicar estas técnicas de forma generalizada.

2.1. Definición de Inteligencia Artificial.

Existen diferentes definiciones de IA, algunas de los autores/científicos más reconocidos en este campo son:

John McCarthy (1956, padre de la IA)

“La Inteligencia Artificial es la ciencia e ingeniería de hacer máquinas inteligentes, especialmente programas de computadora inteligentes.”

Marvin Minsky (1967, cofundador del MIT AI Lab)

*“La Inteligencia Artificial es la ciencia de hacer que las máquinas hagan cosas que requerirían inteligencia **si las hicieran los humanos.**”*

Elaine Rich (1983)

*“La Inteligencia Artificial es el estudio de cómo hacer que las computadoras realicen cosas en las que, por el momento, **las personas son mejores.**”*

Patrick Winston (1992, MIT)

“La Inteligencia Artificial es el estudio de las representaciones computacionales del conocimiento y de los procesos que lo utilizan.”

Stuart Russell y Peter Norvig (1995, Artificial Intelligence: A Modern Approach)

*“La Inteligencia Artificial es el estudio de los **agentes que reciben percepciones del entorno y realizan acciones.**”*

Nilsson (1998)

*“La Inteligencia Artificial concierne a actividades como el pensamiento humano, **el razonamiento, el aprendizaje y la adaptación.**”*

Se puede observar varias perspectivas sobre que se entiende por IA:

- Sistema que se comporta como un humano.
- Sistemas que representan conocimiento y razonamiento.
- Sistemas con capacidad de aprender, adaptarse y actuar en entornos complejos.

2.2. Evolución histórica.

Desde la aparición de los primeros sistemas digitales a mediados/finales de los años 40 del siglo pasado se han producido algunos hitos históricos en la evolución de la IA:

Test de Turing (1950) – Alan Turing

En 1950, el matemático británico Alan Turing publicó *Computing Machinery and Intelligence*, donde formuló la pregunta: “¿*Pueden las máquinas pensar?*”. En lugar de buscar una definición abstracta de inteligencia, propuso el **Test de Turing**, un experimento en el que una máquina es considerada inteligente si puede mantener una conversación escrita con un humano **sin ser descubierta**. Este hito sentó las bases filosóficas y prácticas de la IA, planteando la inteligencia como **comportamiento observable**, más que como un proceso interno.

Conferencia de Dartmouth (1956) – John McCarthy, Marvin Minsky, Allen Newell, Herbert Simon

En 1956 se celebró la Conferencia de Dartmouth, considerada el nacimiento formal de la IA como disciplina. Se estableció que la IA debía abordar problemas de **razonamiento, aprendizaje y resolución de problemas mediante computadoras**. De esta reunión surgieron, entre otras, la lógica **simbólica**, los **algoritmos de búsqueda** y los **sistemas de resolución** general de problemas, base de la IA actual.

Sistemas expertos: MYCIN (1972) – Edward Shortliffe

En los años 70 se popularizaron los sistemas expertos, programas que usaban reglas IF-THEN para imitar el conocimiento de especialistas humanos en dominios específicos. Uno de los más influyentes fue MYCIN, desarrollado por Edward Shortliffe en la Universidad de Stanford en 1972, diseñado para diagnosticar infecciones bacterianas y recomendar antibióticos. **Basado en motores de inferencia y lógica difusa**, representó la aplicación práctica de la IA simbólica en medicina. Aunque limitado en flexibilidad, mostró el potencial de la IA para el **apoyo en la toma de decisiones**.

Deep Blue vence a Kasparov (1997) – IBM

En 1997, la supercomputadora Deep Blue, desarrollada por IBM, venció al campeón mundial de ajedrez Garry Kasparov. Este evento marcó un antes y un después en la percepción pública de la IA. Deep Blue combinaba **algoritmos de búsqueda en árboles** con **técnicas de heurística** y una gran capacidad de **procesamiento paralelo**. Aunque no “pensaba” como un humano, su capacidad para evaluar millones de posiciones por segundo demostró la potencia de la IA basada en fuerza bruta y heurísticas, consolidando el papel de la computación masiva en los avances de la IA.

Procesamiento de Lenguaje Natural: ELIZA (1966) y Watson (2011)

En 1966, Joseph Weizenbaum desarrolló ELIZA, uno de los primeros programas de Procesamiento de Lenguaje Natural (PLN), que simulaba un psicoterapeuta mediante **reglas simples de sustitución de texto**. Aunque muy limitado, ELIZA mostró el potencial de la IA para interactuar con los humanos en su propio idioma. Décadas más tarde, en 2011, IBM Watson ganó el concurso televisivo Jeopardy! enfrentando a campeones humanos, gracias a **técnicas de análisis semántico, minería de textos, búsqueda de información y aprendizaje automático**. Estos hitos representan la evolución del PLN: desde reglas simbólicas básicas hasta sistemas complejos basados en machine learning y big data, sentando el camino hacia los actuales modelos de lenguaje como GPT.

AlexNet y la revolución del Deep Learning (2012) – Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever

En 2012, el modelo **AlexNet**, creado por Alex Krizhevsky bajo la dirección de Geoffrey Hinton e Ilya Sutskever, revolucionó la visión por computadora al ganar el concurso ImageNet con una precisión sin precedentes. Basado en **redes neuronales convolucionales (CNNs)** y potenciado por el **uso de GPUs para entrenamiento masivo**, este avance marcó el inicio del Deep Learning moderno. AlexNet demostró que, con suficientes datos y poder de cómputo, las redes profundas podrían superar a los métodos tradicionales, abriendo la era actual de aplicaciones de IA en visión, voz y texto.

ChatGPT y la era de la IA generativa (2022) – OpenAI

En noviembre de 2022, OpenAI lanzó ChatGPT, un sistema conversacional basado en el modelo GPT-3.5 entrenado con la **arquitectura de Transformers**. Su capacidad para generar texto coherente, mantener diálogos fluidos y adaptarse a múltiples contextos lo convirtió en la aplicación de IA más popular de la historia. Este hito representa la llegada de la IA generativa, no solo en texto, sino también en imágenes, audio y video. Los fundamentos se basan en **modelos de lenguaje de gran escala (LLMs)**, entrenados con billones de parámetros y

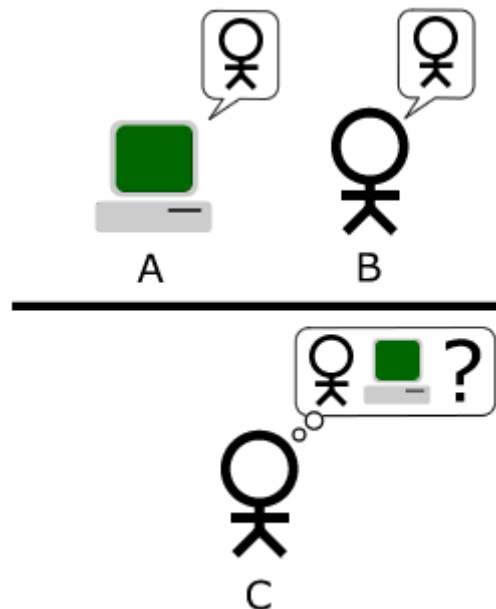
cantidades masivas de datos. ChatGPT democratizó el acceso a la IA, generando tanto entusiasmo como debates éticos y sociales

2.3. Test de Turing.

Uno de los elementos más conocidos de la inteligencia artificial es el Test de Turing, comentado en el punto anterior, propuesto por el matemático Alan Turing, en el ensayo “Computing Machinery and Intelligence” en 1950, que se basa en si una máquina puede hacer pasar por un humano ante otro humano que no conozca si el interlocutor es otro humano o una máquina, interactuando con ella.

Lo novedoso de esta prueba es cambiar la visión “¿pueden las máquinas pensar?” a “¿Pueden las máquinas hacer lo que nosotros (como entidades pensantes) hacemos?”.

La prueba más conocida no definida en el ensayo, sino que fue un debate en un programa de la BBC, en la que el matemático hace que un jurado pregunte a un ordenador, siendo la misión del ordenador engañar al jurado haciéndose pasar por un humano.



Test de Turing.

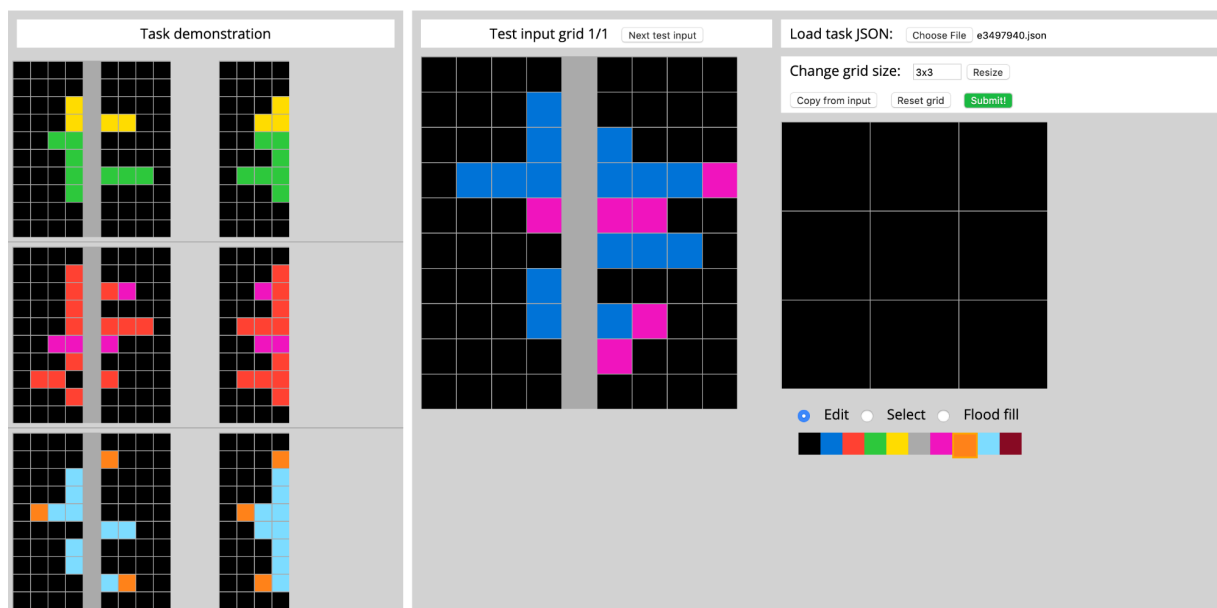
Fuente: https://commons.wikimedia.org/wiki/File:Turing_Test_version_3.png

Autor: <https://commons.wikimedia.org/wiki/User:Bilby>. Licencia: Public Domain

Si bien existen tecnologías que han conseguido pasar el test de forma superficial, en la actualidad no existe una inteligencia artificial capaz de engañar a los jueces de forma generalizada, aunque la aparición de IA's como Chapgt, basados en LLM, han sido un gran avance.

La comunidad científica considera que el Test de Turing no es suficiente para medir la inteligencia artificial definiendo test complementarios o sustitutos como:

- ARC: Mide el razonamiento abstracto y generalizado. Se trata de pequeños puzzles visuales, en el que se debe encontrar reglas ocultas. Mide si una IA es capaz de aprender nuevas reglas sin entrenamiento previo.



Ejemplo de puzzles ARC.

Fuente: <https://github.com/fchollet/ARC-AGI?tab=readme-ov-file>

Autor: <https://github.com/fchollet/ARC-AGI/commits?author=fchollet>. Licencia: [Apache 2.0](#)

- Big-bench: Evaluar capacidades de los modelos de lenguaje grandes (LLMs) en múltiples dimensiones. contiene más de doscientas tareas como puede ser responder a preguntas de cultura general, acertijos, problemas lógicos, matemáticas o traducción.
- MMLU: Evaluar el conocimiento académico y profesional de los modelos de IA. Define tareas sobre biología, física, matemáticas, historia, medicina, ingeniería...

Un ejemplo de MMLU de un "Datasheet" disponible en [Kaggle](#), con licencia GLP :

Which of the following heavenly bodies have never had a spacecraft landed on it?

1. Venus
2. Mars
3. The Moon
4. Jupiter

3. Fundamentos de los sistemas inteligentes.

Las IA se ejecutan en sistemas digitales, en concreto en ordenadores, pero intenta a partir de los transistores, circuitos, chips, microcontroladores, placas, sistemas operativos, lenguajes y herramientas ser capaces de desarrollar inteligencia.

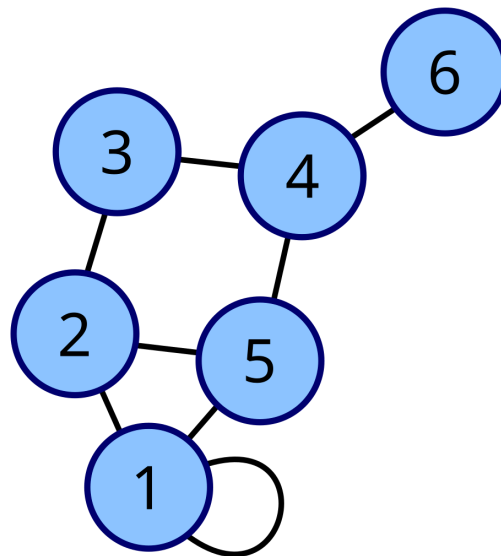
En cada uno de los ámbitos en los que las inteligencias artificiales se han desarrollado, se han desarrollado técnicas basadas en fundamentos matemáticos, biológicos, físicos, cognitivos (especialmente la lingüística) y filosóficos.

Fundamentos Matemáticos.

Los sistemas inteligentes se apoyan fuertemente en las matemáticas para modelar, representar y resolver problemas.

- **Lógica formal:** base de los sistemas basados en reglas, razonamiento deductivo e inferencia.

- **Álgebra lineal:** representación de datos, vectores, tensores y operaciones en redes neuronales.
- **Cálculo diferencial:** optimización de funciones de coste en algoritmos de aprendizaje.
- **Probabilidad y estadística:** razonamiento bajo incertidumbre, aprendizaje bayesiano, inferencia probabilística.
- **Teoría de grafos:** representación de redes neuronales, relaciones entre entidades, búsqueda en grafos.
- **Optimización matemática:** algoritmos de búsqueda, entrenamiento de modelos, planificación.



Grafo no dirigido.

Fuente: <https://commons.wikimedia.org/wiki/File:6n-graph2.svg>

Licencia: Dominio público.

Fundamentos Físicos.

La IA y los sistemas inteligentes también se inspiran en principios de la física:

- **Termodinámica y sistemas dinámicos:** analogía con redes neuronales que buscan estados estables (ej. redes de Hopfield).
- **Teoría de la información (Shannon):** cuantificación de la información y su transmisión, usada en compresión y aprendizaje automático.
- **Mecánica cuántica:** da pie al campo emergente de la computación cuántica y la IA cuántica.
- **Sistemas caóticos y teoría del control:** aplicados en robótica, agentes autónomos y optimización no lineal.

Fundamentos Biológicos.

Gran parte de la IA se inspira en procesos biológicos y cognitivos:

- **Neurociencia:** las redes neuronales artificiales se basan en las conexiones y activaciones de neuronas biológicas.
- **Evolución biológica:** algoritmos genéticos y evolutivos imitan selección natural, mutación y recombinación.

- **Etología (comportamiento animal):** modelos de enjambre (swarm intelligence), como algoritmos de colonias de hormigas o enjambres de partículas.
- **Percepción sensorial:** visión artificial y procesamiento de lenguaje natural imitan cómo los humanos perciben el mundo.

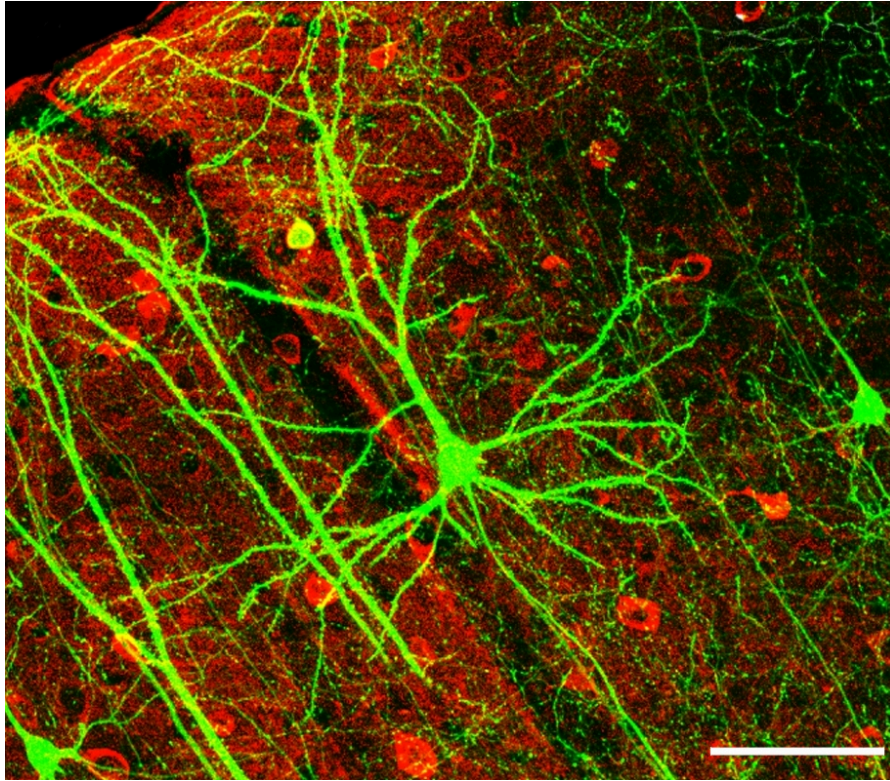


Imagen de una neurona.

Fuente: <https://commons.wikimedia.org/wiki/File:PLoSBiol4.e126.Fig6fNeuron.jpg>

Autor: Wei-Chung Allen Lee. Licencia: Creative Commons

Fundamentos Cognitivos y Filosóficos.

Los sistemas inteligentes también beben de las ciencias cognitivas y la filosofía de la mente:

- **Psicología cognitiva:** modelos de memoria, atención y aprendizaje aplicados en IA cognitiva.
- **Lingüística:** base del procesamiento de lenguaje natural.
- **Epistemología y lógica:** qué significa “conocer” y cómo se representa el conocimiento en sistemas inteligentes.
- **Ética:** desarrollo de IA responsable, control de sesgos y toma de decisiones autónomas.

4. Campos de aplicación.

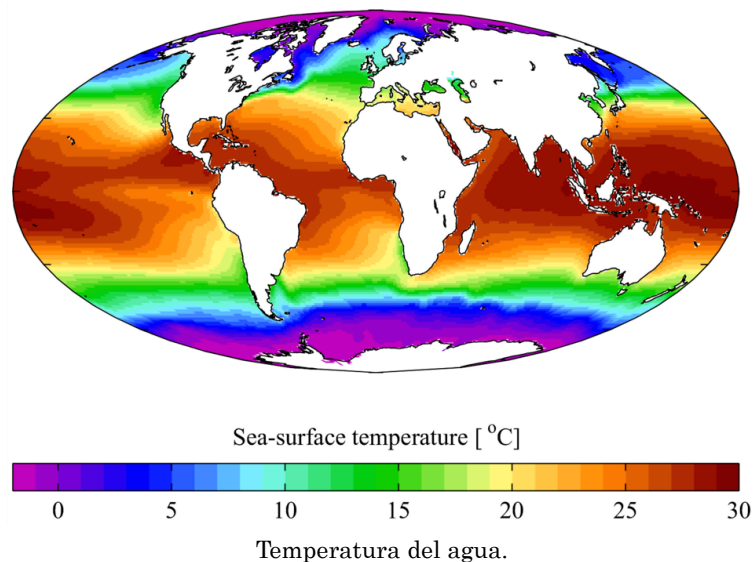
Los campos de aplicación de la IA son prácticamente infinitos, aunque se centra en aquellos en los que son necesarios:

- Analizar grandes volúmenes de datos.
- Identificación de patrones.
- Toma de decisión.

A continuación se detallan algunos de los campos de aplicación más destacados:

Salud. La IA se utiliza para el diagnóstico temprano de enfermedades, la predicción de tratamientos efectivos y la gestión de historiales clínicos. Por ejemplo, en la detección temprana del cáncer de mama o de pulmón, algoritmos de aprendizaje profundo basados en álgebra lineal, cálculo diferencial y estadística bayesiana analizan imágenes médicas. Estos modelos matemáticos permiten identificar anomalías que a veces pasan desapercibidas al ojo humano, mejorando así la tasa de éxito en los tratamientos.

Medio ambiente. En especial la predicción y simulación del cambio climático. Gracias a la IA, los modelos meteorológicos han aumentado en precisión, permitiendo prever fenómenos extremos con más anticipación. En este caso, la base está en las ecuaciones diferenciales parciales de la física de fluidos y la termodinámica atmosférica, combinadas con redes neuronales recurrentes y convolucionales que aprenden de series temporales de datos climáticos. Proyectos globales como el AI for Earth de Microsoft o el Centro Europeo de Predicción Meteorológica a Medio Plazo integran estos fundamentos para simular la evolución del planeta bajo distintos escenarios de emisiones.



Fuente: https://commons.wikimedia.org/wiki/File:Sea_surface_temperature_2001.png

Autor: <https://en.wikipedia.org/wiki/User:Plumbago>

Movilidad y transporte. La IA está transformando la manera en que las personas se desplazan. Los sistemas de vehículos autónomos utilizan sensores y algoritmos de visión por computadora para interpretar el entorno y tomar decisiones en tiempo real. Aquí se aplican fundamentos de la geometría computacional, la teoría de grafos y el álgebra matricial para el procesamiento de imágenes, así como principios de cinemática y dinámica newtoniana para la navegación segura del vehículo. Empresas como Waymo demuestran cómo esta combinación de matemáticas y física aplicada da lugar a sistemas de conducción autónoma más confiables.

Sector de la educación y la comunicación. Personalizando la enseñanza mediante plataformas que adaptan los contenidos al ritmo y estilo de cada estudiante. En traducción automática, herramientas como Google Translate o DeepL utilizan modelos de lenguaje basados en teoría de la información, probabilidad y álgebra tensorial, lo que permite procesar enormes corpus lingüísticos y generar traducciones con mayor fluidez. Asimismo, en la recomendación de contenidos educativos, se emplean técnicas de optimización y análisis de redes.

ingeniería. La inteligencia artificial se aplica en el diseño, control y optimización de sistemas complejos. Por ejemplo, en la ingeniería civil y mecánica, la IA se emplea para analizar estructuras, predecir fallos en materiales y optimizar procesos de fabricación. Se utilizan métodos numéricos, álgebra lineal y cálculo multivariable en combinación con algoritmos de aprendizaje automático que detectan patrones en datos experimentales. En la ingeniería eléctrica, los sistemas de mantenimiento predictivo en redes eléctricas o turbinas eólicas se apoyan en la teoría de señales y sistemas dinámicos, junto con modelos de predicción basados en redes neuronales. Un ejemplo global lo encontramos en proyectos de smart grids (redes eléctricas inteligentes), donde la IA ayuda a equilibrar la demanda y generación de energía renovable.

Desarrollo de software. No solo se usa como herramienta final, sino que también transforma la forma en que se programan y mantienen los sistemas. Los entornos de desarrollo actuales integran asistentes inteligentes que sugieren código, detectan errores y optimizan algoritmos de manera automática. Estos sistemas se apoyan en teoría de autómatas, lógica formal y grafos, que permiten analizar la estructura del código, así como en modelos de lenguaje basados en probabilidad y álgebra tensorial para generar nuevas líneas de programación. Ejemplos globales incluyen herramientas como GitHub Copilot o los sistemas de prueba automática basados en IA que aceleran los ciclos de desarrollo.

La evolución de los campos de aplicación de la IA refleja una expansión desde usos iniciales en entornos industriales y académicos hacia áreas de impacto social y global. Hoy en día, proyectos como Human Brain Project en Europa o OpenAI a nivel internacional no solo buscan potenciar las capacidades de la IA, sino también garantizar un desarrollo ético. Estos esfuerzos se apoyan en la neurociencia computacional, la teoría de sistemas complejos y la lógica matemática para modelar el conocimiento y la toma de decisiones.

5. Técnicas de la Inteligencia Artificial.

A partir de los fundamentos se han desarrollado diferentes técnicas en la IA, tratando en este curso las principales:

5.1. Resolución de problemas.

Resolución de problemas.

Una de las funciones más destacada de la informática es la de encontrar soluciones a problemas complejos en los que existen millones de posibilidades, a todas las posibilidades se le denomina **espacio de estados**, y los algoritmos se encargan de buscar en ese espacio **soluciones válidas o la mejor solución**. El ejemplo clásico es el de calcular el siguiente movimiento en una partida de ajedrez, en el que el espacio puede tener hasta 10^{154} estados (posibles tableros), donde resolverlo por fuerza bruta es imposible con la capacidad de cómputo actual.

Al igual que en cualquier algoritmo la complejidad espacial, pero especialmente temporal son determinantes, que se define en función del tamaño del problema n , en este caso el espacio de búsqueda, siendo de mejor a peor:

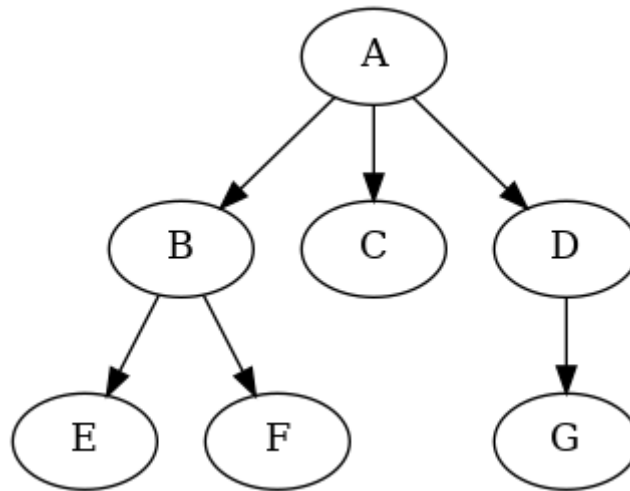
Orden de complejidad	Notación	Descripción	Ejemplo conceptual
Constante	$O(1)$	Tiempo de ejecución independiente del tamaño del problema	Acceder a un elemento de un vector
Logarítmica	$O(\log n)$	Crece muy lentamente al aumentar n	Búsqueda binaria en un vector ordenado
Lineal	$O(n)$	Crece proporcionalmente al tamaño del problema	Recorrer todos los elementos de una lista
Lineal-logarítmica	$O(n \log n)$	Crece un poco más rápido que lineal	Algoritmos de ordenación eficientes (MergeSort, QuickSort promedio)
Cuadrática	$O(n^2)$	Crece con el cuadrado del tamaño del problema	Comparar todos los pares de elementos (Bubble Sort)
Cúbica	$O(n^3)$	Crece con el cubo del tamaño del problema	Multiplicación de matrices clásica
Exponencial	$O(2^n)$	Crece muy rápidamente ; se vuelve impráctico para valores grandes de n	Problemas combinatorios, fuerza bruta en IA
Factorial	$O(n!)$	Crece extremadamente rápido ; impráctico para n medianos	Problema del viajante (TSP) por fuerza bruta

Los principales grupos de algoritmos/técnicas:

5.1.1. Búsquedas no informadas.

No se tiene información de la naturaleza del problema que determine el orden en que se realiza la búsqueda en el espacio de estados, simplemente se dispone de la información del estado inicial y del camino que se ha tomado al estado actual, no del estado actual hasta un estado solución (o aproximación), los algoritmos más conocidos de este grupo son:

Búsqueda en amplitud (BFS). El orden de evaluación de los nodos es por niveles, es decir no, pasa al siguiente nivel hasta que no ha evaluado todos los nodos del nivel actual. Suele utilizarse una cola para encolar y desencolar los nodos/estados. Un ejemplo de recorrido:



Cola inicial: [A]

Desencolar A, imprimir → A

Encolar hijos de A → cola [B,C, D]

Cola: [B, C, D]

Desencolar B, imprimir → B

Encolar hijos de B en → cola [C, D, E, F]

Cola: [C, D, E, F]

Desencolar C, imprimir → C

C no tiene hijos → cola [D, E, F]

Cola [D, E, F]

Desencolar D, imprimir → D

Encolar hijos de D en → cola [E, F, G]

Cola: [E, F, G]

Desencolar E, imprimir → E

E no tiene hijos → cola [F, G]

Cola: [F, G]

Desencolar F, imprimir → F

F no tiene hijos → cola [G]

Cola: [G]

Desencolar G, imprimir → G

G no tiene hijos → cola []

Recorrido: A B C D E F G

Búsqueda en profundidad (DFS). En este caso se recorren antes los nodos hijos, antes que los nodos vecinos. Utiliza la recursión o apila estados de la estructura de datos de tipo pila. Recorre todos los nodos del árbol (recursión) de espacio de estados. Se puede limitar la profundidad del árbol de recursión, denominándose **DFS limitado(DLS)** a esta variación, que mejora el tiempo de ejecución, pero la solución será menos precisa. Un ejemplo de un recorrido sobre el árbol anterior:

El recorrido será, con pila::

Pila inicial: [A]

Sacar A, imprimir → A

Apilar hijos de A en orden inverso → pila [D, C, B]

Pila: [D, C, B]

Sacar B, imprimir → B

Apilar hijos de B en orden inverso → pila [D, C, F, E]

Pila: [D, C, F, E]

Sacar E, imprimir → E

E no tiene hijos → pila [D, C, F]

Pila: [D, C, F]

Sacar F, imprimir → F

F no tiene hijos → pila [D, C]

Pila: [D, C]

Sacar C, imprimir → C

C no tiene hijos → pila [D]

Pila: [D]

Sacar D, imprimir → D

Añadir hijos de D → pila [G]

Pila: [G]

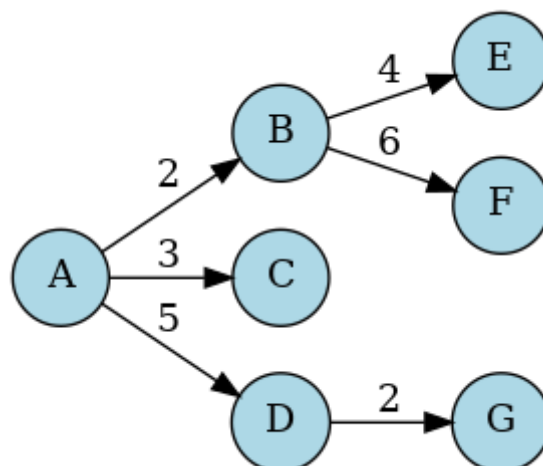
Sacar G, imprimir → G

G no tiene hijos → pila []

Recorrido: A B E F C D G

Búsqueda de coste uniforme (Uniform Cost Search). Este algoritmo selecciona el siguiente nodo/estado que menor coste acumulado tenga. Es necesario por tanto definir cómo calcular el coste acumulado. Es similar a BFS, en cuanto al uso de una cola, en este caso es una cola con **prioridad**, siendo esta prioridad el coste acumulado de cada nodo/estado.

Sea grafo anterior, con pesos asignados a cada arco:



El recorrido UCS, que selecciona el coste acumulado mayor:

Cola inicial: [(0,A)]

Desencolar A, imprimir → A

Encolar hijos de A (observar que se encola por coste) → cola [(5,D), (3,C), (2,B)]

Cola: [(5,D), (3,C), (2,B)]

Desencolar D, imprimir → D

Encolar hijos de D en por coste → cola [(2+5,G), (3,C), (2,B)]

Cola: [(2+5,G), (3,C), (2,B)]

Desencolar G, imprimir → G

G no tiene hijos → cola [(3,C), (2,B)]

Cola [(3,C), (2,B)]

Desencolar C, imprimir → C

C no tiene hijos → cola [(2,B)]

Cola: [(2,B)]

Desencolar B, imprimir → B

Encolar hijos de B → cola [(6+2,F), (4+2,E)]

Cola: [(8,F), (6,E)]

Desencolar F, imprimir → F

F no tiene hijos → cola [(4,E)]

Cola: [E]

Desencolar E, imprimir → E

E no tiene hijos → cola []

Recorrido: A D G C B F E

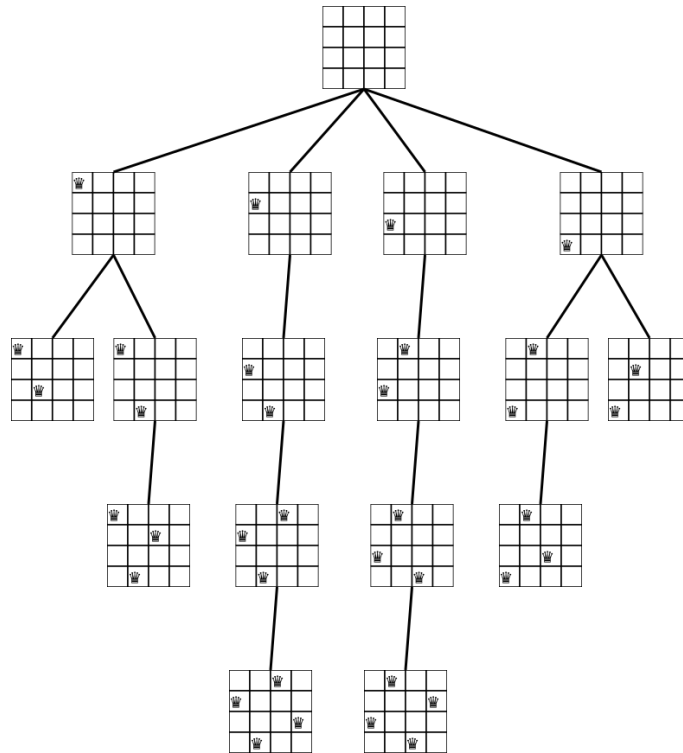
Backtracking. Optimización del algoritmo en profundidad, cuando se busca un estado que cumpla una serie de requisitos, explorar “caminos” que se sabe que no cumplen con los requisitos no tiene mucho sentido, por lo que antes de continuar buscando, se evalúa si el estado actual incumple las condiciones, de ser así, no se tiene en cuenta ese estado para seguir buscando, volviendo hacia atrás en la búsqueda para seguir explorando el espacio de búsqueda.

DFS = estrategia de exploración

Backtracking = DFS + restricciones + retroceso inteligente.

Algunos de los problemas clásicos que resuelve este algoritmo:

- N-Reinas.
- Sudoku.
- Laberintos.



Ejemplo árbol de búsqueda solución para el problema de las 4-Reinas usando Backtracking.

Fuente: https://commons.wikimedia.org/wiki/File:Search_tree_4queen_problem.svg. Licencia: Creative

Commons. Autor: Fschwarzentruber

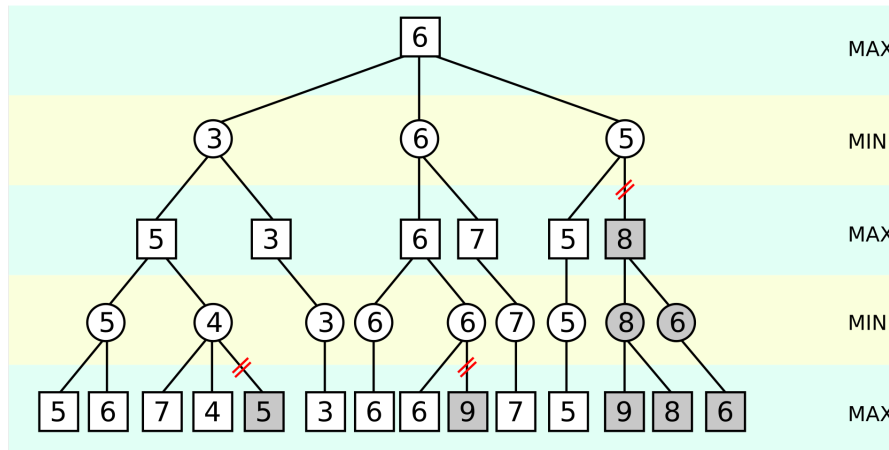
En el caso de los algoritmos anteriores, la complejidad se encuentra en función de diferentes parámetros, como el factor de ramificación (llamadas recursivas) o la profundidad del árbol.

Algoritmo	Complejidad Temporal	Complejidad Espacial	Observaciones
BFS (Breadth-First Search)	$O(b^d)$	$O(b^d)$	Garantiza encontrar la solución más corta (en número de pasos). Pero consume mucha memoria porque guarda todos los nodos del nivel actual.
DFS (Depth-First Search)	$O(b^m)$	$O(b \cdot m)$	No garantiza una solución óptima. Espacio lineal en la profundidad máxima, mucho menor que BFS.
Uniform Cost Search (UCS)	$O(b^{(1+LC^*/\epsilon \lceil \log \frac{C^*}{\epsilon} \rceil)})$	$O(b^{(1+LC^*/\epsilon \lceil \log \frac{C^*}{\epsilon} \rceil)})$	Expande siempre el nodo de menor coste acumulado . Encuentra la solución óptima en coste , pero puede ser muy caro en tiempo y espacio.
Backtracking	$O(b^m)$ en el peor caso	$O(b \cdot m)$	DFS con poda por restricciones . Mucho más eficiente en práctica si las podas eliminan ramas enteras. Usado en N-Reinas, Sudoku, CSP.

Mencionar que existen algoritmos de búsqueda no informada especializados en juegos, por ejemplo, son famosos los algoritmos para encontrar soluciones en juegos por turnos de dos jugadores, como son el Mix-Max o el Alpha-Beta.

Min-Max: Algoritmo fundamental en juegos de dos jugadores, como el ajedrez o tres en raya. Su objetivo es encontrar la jugada óptima para un jugador, llamado MAX, bajo el supuesto de que el oponente, llamado MIN, también juega de manera óptima. El problema de este es su coste exponencial, ya que es necesario desarrollar todo el espacio de estados.

Alpha-Beta: Técnica que evita explorar ramas del árbol que no afectarán la decisión final. Esta estrategia mantiene dos valores: α (alpha), el mejor valor garantizado para MAX hasta el momento, y β (beta), el mejor valor garantizado para MIN. Si durante la evaluación se encuentra que $\alpha \geq \beta$, significa que continuar explorando esa rama no cambiará la decisión del jugador actual, por lo que la rama se poda y no se analiza más.



Ejemplo de algoritmo Alpha-Beta, con poda de ramas.

Fuente: https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning#/media/File:AB_pruning.svg. Licencia: Creative Commons. Autor: Jez9999

5.1.2. Búsqueda informada.

Estos algoritmos se basan en la información de estado actual y de información adicional denominada heurística (técnica de la indagación y del descubrimiento), de forma que la decisión del siguiente nodo a explorar se toma en función de esta información(n).

Una heurística será mejor o peor en función de medir cuán cerca está la solución del nodo. Los principales algoritmos de búsqueda basados en heurística:

Algoritmo	Estrategia	Complejidad temporal/espacial (aprox.)	Propiedades
Greedy Best-First Search	Expande el nodo con menor heurística $h(n)$.	Exponencial en el peor caso: $O(b^m)$.	No siempre encuentra la solución óptima (puede caer en caminos malos). Rápido si la heurística es buena.

A*	Expande el nodo con menor $f(n)=g(n)+h(n)$, donde $g(n)$ es el coste real desde el inicio, y $h(n)$ es la heurística.	Exponencial en el peor caso: $O(b^d)$, pero mucho mejor que búsqueda ciega si $h(n)$ es buena.	Si la heurística es admisible (no sobreestima), garantiza encontrar la solución óptima.
IDA* (Iterative Deepening A*)	Variante de A* que usa límites iterativos sobre $f(n)$ para reducir espacio.	Tiempo: peor que A*. Espacio: solo $O(b \cdot d)$.	Reduce el uso de memoria, muy usado en problemas como el 8-puzzle o ajedrez.
Heuristic Local Search	Buscan directamente sobre el espacio de estados con evaluaciones heurísticas locales.	Depende del problema.	No garantizan encontrar la solución óptima, pero útiles en espacios enormes.

5.2. Representación e inferencia.

Otro grupo de técnicas que forma parte de la IA es la de cómo representar del conocimiento y cómo a partir de esta representación razonar (inferir).

La Wikipedia define el conocimiento como: “La familiaridad, la consciencia o la comprensión de alguien o de algo, como pueden ser los hechos (conocimiento descriptivo), las habilidades (conocimiento procedimental) o los objetos (conocimiento por familiaridad). En la mayoría de los casos, el conocimiento puede adquirirse de muchas maneras y a partir de muchas fuentes, como la percepción, la razón, la memoria, el testimonio, la investigación científica, la educación y la práctica.”

Se puede clasificar el conocimiento en:

Tipo de conocimiento	Qué representa	Cómo se infiere	Ejemplos
Declarativo	Hechos y relaciones sobre el mundo	Deducción lógica, reglas	“Todos los humanos son mortales”, “El Sol es una estrella”
Procedimental	Cómo hacer algo (procedimientos o algoritmos)	Ejecución de pasos secuenciales o condicionales	Cómo sumar números, cómo mover un robot hacia un objetivo
Heurístico	Reglas empíricas o aproximaciones para guiar la búsqueda	Algoritmos informados (A*, Greedy, Hill-Climbing)	En ajedrez: “Controla el centro del tablero”, laberintos: “Avanza hacia la salida”

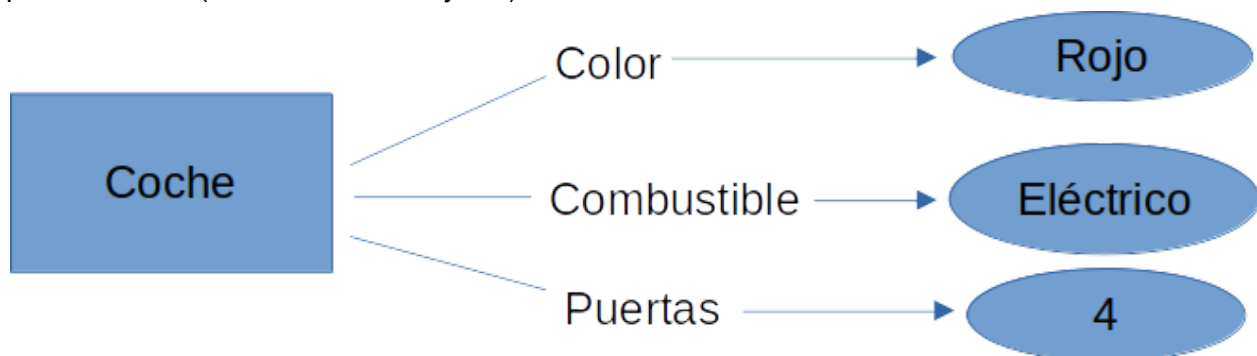
Estratégico / Experto	Estrategias y tácticas específicas de un dominio	Encadenamiento hacia adelante o hacia atrás, reglas de sistemas expertos	Diagnóstico médico, estrategias de negocio
Probabilístico / Incierto	Conocimiento con incertidumbre o probabilidades	Cálculo probabilístico, razonamiento bayesiano, propagación de incertidumbre	“Si hay nubes, 70% probabilidad de lluvia”, redes bayesianas para diagnóstico

5.2.1. Representación del conocimiento.

Para que los sistemas puedan dar soluciones a los problemas y cuestiones que se les plantea en primer lugar se ha de almacenar el conocimiento sobre el que posteriormente se trabajará. Existen diferentes técnicas de representación de información:

Objeto-Atributo-Valor (OAV)

La representación Objeto-Atributo-Valor organiza el conocimiento en términos de entidades (objetos), sus propiedades (atributos) y los valores que estos atributos pueden tomar. Por ejemplo, un objeto “Coche” puede tener atributos como color, modelo o velocidad, con valores “rojo”, “2025” o “120 km/h”. Esta técnica permite estructurar información de manera modular y facilita la inferencia mediante la herencia de atributos y la propagación de valores a través de objetos relacionados. Es útil tanto para conocimiento declarativo (hechos sobre objetos) como procedimental (acciones sobre objetos).



Hechos inciertos / Probabilísticos

En muchos problemas del mundo real, la información es imprecisa o incierta. Los hechos probabilísticos representan conocimiento con grados de certeza, usando probabilidades o modelos estadísticos. Por ejemplo, “Si hay nubes, hay un 70% de probabilidad de lluvia”. La inferencia se realiza mediante razonamiento probabilístico, redes bayesianas o propagación de incertidumbre. Este tipo de representación es fundamental para el conocimiento probabilístico, donde no se puede garantizar certeza absoluta, y también se puede usar para guiar decisiones heurísticas.

Conocimiento difuso (Fuzzy)

El conocimiento difuso permite representar información imprecisa o gradual, en contraste con los valores estrictamente verdaderos o falsos de la lógica clásica. Por ejemplo, se puede definir que la temperatura es “alta” con un grado de 0.7 y “moderada” con 0.3. La inferencia se realiza usando reglas difusas y operadores de lógica fuzzy, combinando valores parciales para tomar decisiones. Este enfoque es útil en sistemas donde los conceptos son subjetivos o vagos, como control de clima, electrodomésticos inteligentes o diagnósticos médicos, y se asocia a conocimiento heurístico o probabilístico.

Reglas (If-Then)

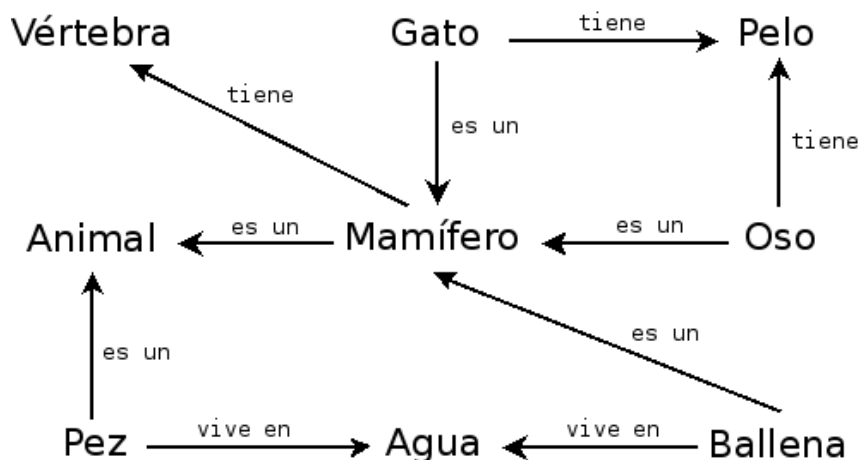
Los sistemas basados en reglas representan conocimiento como un conjunto de condiciones y acciones del tipo “Si... Entonces...”. Por ejemplo, “Si la temperatura > 100°C, entonces hervir agua”. La inferencia puede realizarse mediante encadenamiento hacia adelante (partiendo de hechos conocidos) o hacia atrás (partiendo de una meta). Esta técnica es muy utilizada en sistemas expertos y permite modelar tanto conocimiento declarativo (hechos y relaciones) como estratégico o procedimental (planes y decisiones basadas en reglas).

Blackboards

El enfoque de blackboard consiste en un espacio compartido donde diferentes módulos especializados aportan conocimiento parcial sobre un problema. Cada módulo puede leer y escribir en la “pizarra” (blackboard), coordinando el razonamiento. La inferencia se realiza de manera colaborativa, donde los módulos contribuyen a derivar nuevas conclusiones según reglas o estrategias locales. Es útil para conocimiento estratégico o procedimental, sobre todo en sistemas complejos como reconocimiento de voz, diagnóstico médico o planificación robótica.

Redes semánticas

Las redes semánticas representan conocimiento mediante nodos (conceptos) y aristas (relaciones entre conceptos). Por ejemplo, “Perro” → es un → “Animal”. La inferencia se realiza propagando información a través del grafo, utilizando herencia y relaciones transitivas. Este enfoque permite responder preguntas sobre jerarquías y relaciones entre conceptos y se asocia principalmente al conocimiento declarativo, describiendo hechos y conexiones entre entidades.



Red semántica. Fuente:

https://es.wikipedia.org/wiki/Red_sem%C3%A1ntica#/media/Archivo:Red_Semantica.png .

Licencia: Dominio Público. Autor: Fernando Sánchez de Arma

Frames (Marcos)

Los frames son estructuras tipo objeto que contienen slots (atributos) y valores, y pueden heredar información de frames más generales. Por ejemplo, un frame "Vehículo" puede tener un atributo ruedas=4, y un frame "Coche" hereda este atributo automáticamente. La inferencia se realiza mediante herencia de atributos y reglas asociadas a slots. Esta técnica combina aspectos declarativos (descripción de objetos y propiedades) y procedimentales (acciones asociadas a slots o eventos).

Lógica proposicional

La lógica proposicional representa hechos simples como proposiciones verdaderas o falsas, usando conectivos lógicos (AND, OR, NOT, $\rightarrow$). Por ejemplo, Llueve $\rightarrow$ SueloMojado. La inferencia se realiza mediante reglas de deducción como modus ponens o resolución.

Es adecuada para representar conocimiento declarativo básico y es la base de muchos sistemas expertos que requieren razonamiento lógico simple. Los elementos básicos:

Elemento	Símbolo	Descripción	Ejemplo
Proposición atómica	$p, q, r, \dots$	Afirmación indivisible que es verdadera o falsa.	p : "Llueve"
Negación	$\neg p$	"No es el caso que..."	Si p : Llueve, entonces $\neg p$: No llueve.
Conjunción	$p \wedge q$	"p y q". Solo verdadera si ambas lo son.	p : Llueve $\wedge$ q : Hace frío
Disyunción	$p \vee q$	"p o q". Es verdadera si al menos una lo es.	p : Llueve $\vee$ q : Hace calor
Implicación	$p \rightarrow q$	"Si p entonces q". Solo falsa si p es verdadera y q es falsa.	p : Estudio $\rightarrow$ q : Apruebo
Doble implicación	$p \leftrightarrow q$	"p si y solo si q". Verdadera cuando ambas tienen el mismo valor de verdad.	p : 2 es par $\leftrightarrow$ q : 2 es divisible entre 2
Constantes lógicas	$\top, \perp$	$\top$: verdadero siempre. $\perp$: falso siempre.	$p \wedge \top = p$

Cálculo de predicados (lógica de primer orden)

La lógica de primer orden extiende la lógica proposicional permitiendo objetos, relaciones y cuantificadores ($\forall, \exists$). Por ejemplo: $\forall x (Humano(x) \rightarrow Mortal(x))$. La inferencia se realiza mediante técnicas como resolución, unificación y deducción generalizada, lo que permite razonar sobre hechos complejos y relaciones. Esta técnica es muy potente para conocimiento

declarativo, especialmente en sistemas expertos que requieren razonamiento formal y generalización sobre múltiples entidades. Los elementos de la lógica de primer orden son:

Elemento	Símbolo/Ejemplo	Descripción
Constantes	a, b, c, juan, 3	Representan objetos concretos en el dominio.
Variables	x, y, z	Representan objetos arbitrarios del dominio.
Funciones	f(x), padre(x), suma(x,y)	Asignan un objeto del dominio a partir de otros objetos.
Predicados	P(x), Q(x,y), hermano(x,y)	Expresan propiedades de objetos o relaciones entre objetos.
Cuantificadores	$\forall x$, $\exists x$	Universal ("para todo") y existencial ("existe al menos uno").
Conectores lógicos	$\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$	Negación, conjunción, disyunción, implicación, equivalencia.
Símbolos de igualdad	=	Relación de identidad entre términos.
Fórmulas atómicas	P(x), Q(a,b)	Expresiones básicas sin conectores ni cuantificadores.
Fórmulas compuestas	$\forall x (P(x) \rightarrow Q(x))$	Se construyen a partir de atómicas usando cuantificadores y conectores.

Un resumen de las técnicas, junto con el tipo de conocimiento asociado:

Técnica de representación	Qué representa	Tipo de conocimiento asociado	Cómo se infiere
Objeto-Atributo-Valor (OAV)	Entidades con propiedades y valores	Procedimental, Declarativo	Herencia de atributos, propagación de valores
Hechos inciertos / probabilísticos	Información con incertidumbre	Probabilístico	Razonamiento probabilístico, redes bayesianas
Conocimiento difuso (fuzzy)	Conceptos imprecisos o graduales	Probabilístico / Heurístico	Inferencia difusa, reglas difusas, lógica fuzzy

Reglas (If-Then)	Conocimiento basado en condiciones y acciones	Declarativo, Estratégico / Experto	Encadenamiento hacia adelante (forward) o hacia atrás (backward)
Blackboards	Conocimiento compartido entre módulos especializados	Procedimental, Estratégico / Experto	Coordinación de agentes y reglas, inferencia basada en control central
Redes semánticas	Conceptos y relaciones entre ellos	Declarativo	Herencia, propagación de información, transitive closure
Frames	Objetos con slots y valores, herencia	Declarativo, Procedimental	Herencia, reglas en slots, encadenamiento
Lógica proposicional	Hechos simples, verdadero/falso	Declarativo	Deducción lógica, modus ponens
Cálculo de predicados (lógica de primer orden)	Objetos, relaciones, cuantificadores	Declarativo	Resolución, unificación, deducción generalizada

5.2.2. Inferencia.

Se puede definir inferencia en IA como el proceso de utilizar un modelo de inteligencia artificial previamente entrenado para realizar predicciones o tomar decisiones sobre datos nuevos y nunca antes vistos.

La representación del conocimiento y la inferencia están estrechamente ligadas: la forma en que se organiza el conocimiento (ya sea en lógica, redes semánticas, marcos ...) determina las estrategias de inferencia posibles. Por ejemplo, en una base de conocimiento expresada en lógica de primer orden, se aplican reglas de deducción.

Las técnicas de inferencia más conocidas son:

Deducción. Es el tipo de inferencia más clásico en Inteligencia Artificial y está basada en la lógica formal. Consiste en partir de principios generales o reglas conocidas y, mediante la aplicación de razonamiento lógico, llegar a conclusiones particulares. Por ejemplo, si se sabe que "todos los humanos son mortales" y que "Sócrates es humano", se puede deducir que "Sócrates es mortal". Este razonamiento es válido siempre que las premisas sean verdaderas y la estructura lógica esté correctamente construida. Algunas reglas de inferencia por deducción:

Regla	Forma	Descripción
Modus Tollens	$p \rightarrow q, \sim q \Rightarrow \sim p$	Si de p se sigue q, y q es falso, entonces p también es falso.
Silogismo hipotético	$p \rightarrow q, q \rightarrow r \Rightarrow p \rightarrow r$	Si p implica q, y q implica r, entonces p implica r.

Silogismo disyuntivo	$p \vee q, \sim p \Rightarrow q$	Si ocurre "p o q" y p es falso, entonces q es verdadero.
Generalización universal	$P(x) \Rightarrow \forall x P(x)$	Si P(x) es válido para un objeto arbitrario, se generaliza a todos los objetos.
Instanciación existencial	$\exists x P(x) \Rightarrow P(c)$ (para algún c)	Si existe un x que cumple P(x), entonces hay al menos un objeto concreto c que cumple P(c).
Instanciación universal	$\forall x P(x) \Rightarrow P(c)$	Si P(x) es cierto para todo x, entonces se cumple para cualquier constante c.

$\sim$ = negación, $\vee$ = disyunción (OR), $\&$ = conjunción (AND), $\rightarrow$ = implicación, $\forall$ = para todo, $\exists$ = existe.

Inducción. La inducción es un proceso inferencial en el que se pasa de casos particulares a generalizaciones. A diferencia de la deducción, sus conclusiones no son necesariamente verdaderas, sino probables, ya que se construyen a partir de la observación de ejemplos. Un caso clásico sería: "He visto cien cisnes y todos eran blancos, por lo tanto, todos los cisnes son blancos". Existen diferentes técnicas de inducción como las basadas en reglas del tipo **si..entonces**, árboles de decisión, inducción lógica,

Refutación. Método de razonamiento lógico que trata de probar que una afirmación es verdadera demostrando que su negación lleva a una contradicción. Es una forma de deducción indirecta, también conocida como reducción al absurdo, muy utilizada en lógica matemática y sistemas expertos. Para aplicar la refutación, se toma la negación de la hipótesis, se combina con los hechos y reglas disponibles, y se busca una inconsistencia; si se encuentra, se concluye que la hipótesis original es verdadera.

Forward chaining (encadenamiento hacia adelante). Parte de un conjunto de hechos conocidos y aplica reglas de la forma "si-entonces" para derivar nuevos hechos hasta alcanzar una conclusión o no poder avanzar más. Por ejemplo, si en la base de conocimiento se tiene "Si un animal es perro, entonces es mamífero" y "Fido es un perro", el sistema deduce "Fido es un mamífero".

Esta técnica es común en motores de reglas y sistemas expertos, donde el conocimiento se representa en forma de producciones (reglas condicionales). Su ventaja es que explora todas las posibles conclusiones a partir de los hechos, aunque puede generar muchos resultados intermedios no necesarios.

Backward chaining (encadenamiento hacia atrás). A diferencia del forward chaining, comienza con una hipótesis o meta que se desea probar y busca en la base de conocimiento las reglas que podrían justificar. Si la conclusión depende de otras condiciones, se intenta demostrar esas condiciones recursivamente. Por ejemplo, si se quiere demostrar que "Fido es un mamífero", el sistema buscará una regla del tipo "si un animal es perro, entonces es mamífero" y verificará si se cumple que "Fido es un perro".

Este método es muy eficiente cuando se tienen hipótesis concretas que comprobar, ya que evita explorar conclusiones irrelevantes. Es ampliamente utilizado en sistemas basados en

lógica, y en asistentes de diagnóstico, donde el conocimiento se representa mediante reglas lógicas y el razonamiento se centra en comprobar metas específicas.

5.3. Procesamiento del lenguaje natural.

Una de las ramas que mayor uso ha tenido en los últimos años, permite a las máquinas entender, interpretar, generar y responder al lenguaje humano, usando el texto y la voz.

Dentro del sistema de procesamiento del lenguaje natural o PLN, existen diferentes fases o etapas para realizar su función, y dentro de estas, se han desarrollado diferentes técnicas.

Las fases son:

1. **Adquisición de datos.** Obtener el texto o habla que se va a procesar. Se prepara el **corpus de datos** para las siguientes fases. (base de datos estructurada que contiene ejemplos reales del uso del lenguaje, cuidadosamente recopilados y organizados para su análisis y procesamiento automáticos.)
2. **Preprocesamiento.** Limpiar y normalizar los datos para que sean útiles para los algoritmos. Algunas de las técnicas son la tokenización (separar el texto), lematización (reducir las palabras a su raíz), eliminar palabras que no aportan información (el, de...), normalización, y corrección ortográfica.
3. **Análisis lingüístico:** Una de las fases más importantes, su función es la de extraer información estructurada del texto. Se divide a su vez en subfases que emplean técnicas muy especializadas:
 - 3.1. Análisis morfológico: Identificar raíces, sufijos y prefijos.
 - 3.2. Sintaxis: Analizar la estructura gramatical de las oración. Un ejemplo de análisis sintáctico para la oración "María lee un libro" es identificar "María" como el Sujeto y "lee un libro" como el Predicado. Dentro del predicado, "lee" es el núcleo del verbo y "un libro" es el Complemento Directo (CD).
 - 3.3. Semántica. Su función es entender el significado de las palabras y frases teniendo en cuenta el contexto. Un ejemplo de análisis semántico es el estudio del significado de la frase "Esa manzana es tan dulce", donde el análisis semántico puede determinar si se refiere a una fruta o a la marca de una empresa, como Apple, basándose en el contexto y las características de las palabras.
 - 3.4. Pragmática: Interpretación del sentido según contexto o intención.

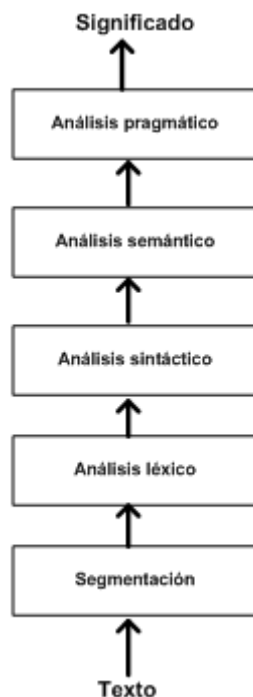


Diagrama de fase 1 (Segmentación y léxico), fase 3 (sintáctico, semántico y pragmático).
 Fuente: https://ca.wikipedia.org/wiki/Programaci%C3%B3_neuroling%C3%BC%C3%ADstica#/media/Fitxer:Fases_PLN.png, Licencia: Dominio Público. Autor: Pancho Alvarez.

4. **Representación del conocimiento:** Convertir el texto en una forma que los algoritmos puedan procesar. Algunas de las técnicas se basan en: Asignar pesos a las palabras según importancia, transformarlas a vectores para medir las distancias entre ellas,
5. **Procesamiento o análisis avanzado:** Aplicar técnicas de IA para resolver tareas específicas. Puede clasificar texto, extracción de entidades o traducción automática.
6. **Postprocesamiento y generación:** Presentar resultados al usuario o generar lenguaje natural.

Un ejemplo sencillo de PLN para interpretar la frase ¿Cuál es la capital de España?

Fase del PLN	Qué hace	Ejemplo con la frase
Adquisición de datos	Obtener el texto o la voz que se va a procesar	Se recibe la entrada del usuario: "¿Cuál es la capital de España?"
Preprocesamiento	Limpiar y normalizar los datos	- Tokenización: ["¿", "Cuál", "es", "la", "capital", "de", "España", "?"] - Eliminación de signos de puntuación: ["Cuál", "es", "la", "capital", "de", "España"] - Lematización: ["cuál", "ser", "la", "capital", "de", "España"]

Análisis lingüístico	Entender la estructura y significado del texto	- Sintaxis: identifica sujeto, verbo y objeto de la pregunta - Semántica: detecta que se busca el nombre de una ciudad que es capital de un país
Representación del conocimiento	Convertir texto a forma que la máquina pueda procesar	- Vectorización: ["capital"]=1, "España "=1, "cuál "=1] - Embeddings: convierte cada palabra en vectores numéricos que capturan significado
Procesamiento / análisis avanzado	Aplicar algoritmos de IA para resolver la tarea	- Sistema de preguntas y respuestas busca en la base de conocimiento que la capital de España es "Madrid"
Postprocesamiento / generación	Formatear o generar la respuesta para el usuario	- Generación de respuesta: "La capital de España es Madrid."
Salida al usuario	Entregar el resultado final	El usuario recibe la respuesta clara y natural: "La capital de España es Madrid."

5.3.1. Sistemas expertos.

Uno de los ámbitos en el que las empresas están invirtiendo más. Su misión es emular el razonamiento de un experto humano en un dominio específico. Su objetivo es resolver problemas complejos mediante el uso de una base de conocimiento y un motor de inferencia, en lugar de depender solo de algoritmos clásicos.

Los sectores en los que emplean los sistemas expertos son múltiples, destacando la medicina, ingeniería, diagnóstico de fallos, planificación, asesoría técnica y toma de decisiones.

En estos sistemas expertos se utilizan muchos de los principios y tecnologías vistas en puntos anteriores, como la lógica difusa o probabilidad.

Un sistema experto se compone de diferentes elementos, cada uno de los cuales se puede construir usando diferentes principios.

Base de conocimiento: Contiene los hechos y reglas sobre el dominio (por ejemplo, diagnóstico de enfermedades, reglas de negocio, etc.). Incluye conocimiento declarativo (qué se sabe) y procedimental (cómo actuar). Puede implementarse entre otras con:

- Lógica proposicional y de primer orden.
- Reglas de producción (if-then).
- Frames y redes semánticas.

Motor de inferencia: Es el elemento que infiere a partir de la base de conocimiento del sistema. Aplica reglas de la base de conocimiento a los hechos actuales para obtener conclusiones.

Se basa en:

- Basados en reglas (sistemas de producción).
- Basados en lógica formal.
- Basados en probabilidades (ej. redes bayesianas).
- Basados en fuzzy logic para manejar incertidumbre.

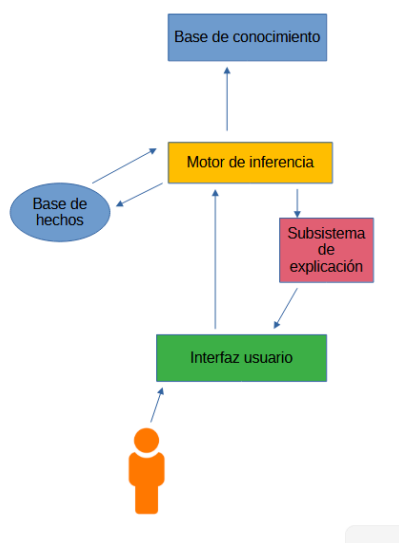
Puede usar para inferir, entre otras técnicas:

- Inducción.
- Deducción.
- Encadenamiento hacia adelante (forward chaining): parte de hechos conocidos y deriva conclusiones.
- Encadenamiento hacia atrás (backward chaining): parte de una hipótesis y busca pruebas que la confirmen.
-

Base de hechos (memoria de trabajo). Contiene los datos específicos del problema actual (datos aportados por el usuario o sensores). Va cambiando durante el proceso de inferencia. Representa el estado dinámico del sistema (cambia en el tiempo). Se puede implementar con listas, grafos, árboles, bases de datos o sistemas de reglas

Interfaz de usuario: Permite la comunicación entre el sistema y el usuario. Puede ser un diálogo de preguntas/respuestas o una interfaz gráfica más avanzada.

Subsistema de explicación: Explica al usuario por qué se tomó cierta decisión (“trazabilidad del razonamiento”). Ejemplo: “El paciente tiene fiebre y dolor de garganta, por lo tanto, podría tener infección bacteriana”.



5.4. Robótica.

Aunque no sea una técnica propia de la IA, si existe una fuerte interacción entre las técnicas anteriores y la robótica, como puede ser el uso de visión por computador.

No existe un único tipo de robot, siendo el ámbito en que se utilizan los que determinan sus características y tecnologías. Los principales tipos de robots:

Tipo de robot	Descripción	Ejemplos	Aplicaciones
Robots industriales	Robots diseñados para tareas repetitivas en fábricas, generalmente brazos robóticos fijos.	KUKA, ABB, FANUC	Ensamblaje, soldadura, pintura, empaquetado
Robots móviles	Se desplazan de manera autónoma o teleoperada en distintos entornos.	AMRs (robots móviles autónomos), AGVs (vehículos guiados automáticos)	Logística, transporte interno, exploración
Robots humanoides	Con apariencia y/o comportamiento similar al humano, capaces de interactuar socialmente.	ASIMO (Honda), Atlas (Boston Dynamics), Sophia (Hanson Robotics)	Investigación, asistencia, interacción social
Robots de servicio	Ayudan en tareas domésticas o de atención al público, fuera del ámbito industrial.	Roomba, Pepper, robots camareros	Limpieza, atención al cliente, logística ligera
Robots médicos	Diseñados para asistir en cirugía, rehabilitación o diagnóstico.	Da Vinci, exoesqueletos médicos	Cirugía asistida, rehabilitación, diagnóstico clínico
Drones / Aéreos	Robots aéreos no tripulados, autónomos o controlados remotamente.	DJI Phantom, drones militares	Fotografía aérea, vigilancia, agricultura, rescate
Robots submarinos	Operan en ambientes acuáticos, autónomos o controlados.	ROVs, AUVs	Exploración marina, investigación, reparación submarina
Nanorrobots / Microrrobots	Robots a micro o nanoescala, inspirados en biología.	Nanobots experimentales	Medicina (cirugía celular, liberación de fármacos)
Cobots (colaborativos)	Robots diseñados para trabajar junto a humanos de forma segura.	UR3e, UR5e, Baxter	Fábricas, ensamblaje, logística colaborativa

Soft robots (robótica blanda)	Robots hechos de materiales flexibles, inspirados en organismos vivos.	Tentáculos robóticos, robots-pulpo	Medicina, manipulación delicada, rescates
--	--	------------------------------------	---

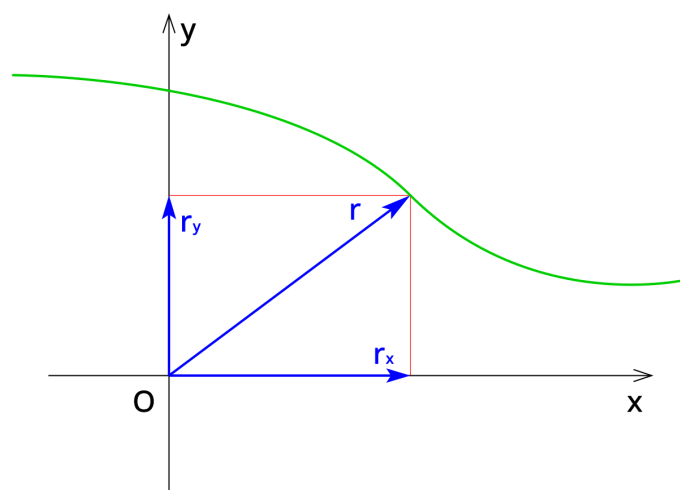
Al igual que las técnicas de inteligencia artificial, en la robótica existen multitud de áreas o ramas, muchas de ellas relacionadas con la IA. Las principales ramas de la robótica son:

Rama de la robótica	Ejemplos	Aplicaciones	Técnicas IA aplicadas
Robótica industrial	Brazos ABB, KUKA, FANUC	Ensamblaje, soldadura, pintura, empaquetado	- Búsqueda en problemas: planificación de trayectorias y optimización. - Inferencia: diagnóstico de fallos. - Representación del conocimiento: modelos de procesos de fabricación. - Sistemas expertos: control de calidad y mantenimiento predictivo.
Robótica de servicio	Roomba, robots camareros, Pepper	Limpieza, logística, atención al cliente	- Búsqueda en problemas: navegación en entornos dinámicos. - Inferencia: decisiones de interacción con usuarios. - Representación del conocimiento: mapas de entornos y perfiles de usuario. - Sistemas expertos: asistencia al cliente y recomendaciones.
Robótica médica	Da Vinci, exoesqueletos	Cirugía, diagnóstico, rehabilitación	- Búsqueda en problemas: planificación de movimientos quirúrgicos. - Inferencia: apoyo en diagnóstico médico. - Representación del conocimiento: bases médicas y modelos anatómicos. - Sistemas expertos: apoyo en toma de decisiones clínicas.

Robótica móvil autónoma	Coches autónomos, drones	Transporte, vigilancia, exploración	- Búsqueda en problemas: planificación de rutas y SLAM. - Inferencia: predicción de comportamiento de obstáculos. - Representación del conocimiento: mapas y modelos del entorno. - Sistemas expertos: gestión de tráfico y control autónomo.
Robótica humanoide	ASIMO, Atlas, Sophia	Interacción social, investigación	- Búsqueda en problemas: planificación de movimientos. - Inferencia: razonamiento sobre intenciones humanas. - Representación del conocimiento: bases semánticas y frames. - Sistemas expertos: asistentes conversacionales y de apoyo social.
Robótica espacial y submarina	Rover Perseverance, ROVs	Exploración espacial y oceánica	- Búsqueda en problemas: exploración en entornos desconocidos. - Inferencia: decisiones autónomas en condiciones extremas. - Representación del conocimiento: mapas de terreno y entornos submarinos. - Sistemas expertos: diagnóstico remoto y control autónomo.
Microrrobótica / Nanorrobótica	Nanobots médicos, microrobots	Medicina, manipulación de materiales	- Búsqueda en problemas: movimiento dentro del cuerpo humano. - Inferencia: respuesta ante condiciones médicas. - Representación del conocimiento: modelos biológicos a microescala. - Sistemas expertos: soporte en terapias dirigidas.

Robótica colaborativa (cobots)	UR3e, UR5e	Trabajo en fábricas junto a humanos	- Búsqueda en problemas: asignación de tareas entre humano y robot. - Inferencia: detección de riesgos y adaptación en tiempo real. - Representación del conocimiento: modelos de colaboración humano-robot. - Sistemas expertos: gestión de seguridad y coordinación.
Robótica blanda (Soft Robotics)	Tentáculos robóticos, robots inspirados en pulpos	Medicina, manipulación delicada	- Búsqueda en problemas: adaptación de movimientos flexibles. - Inferencia: control bajo incertidumbre. - Representación del conocimiento: modelos bioinspirados. - Sistemas expertos: control en entornos no estructurados.

En prácticamente todos los tipos de robots, una de las áreas básicas es la **cinemática** (rama que estudia el movimiento de los robots en términos de posición, orientación, velocidad y aceleración, sin tener en cuenta las fuerzas que producen dicho movimiento).



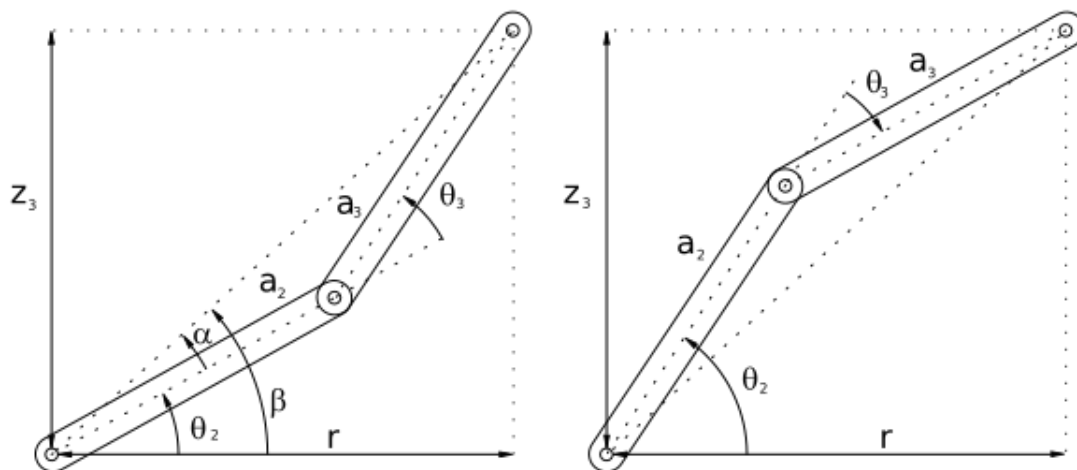
Representación de la trayectoria de una partícula (verde), mostrando la posición (azul) en un momento dado de dicha trayectoria.

Fuente: https://es.wikipedia.org/wiki/Cinem%C3%A1tica#/media/Archivo:Posici%C3%B3n_2D.svg. Licencia:

Creative Commons. Autor: Dnu72

Los problemas principales a los que se enfrenta la robótica en relación a la cinemática son:

- Cinemática directa: Se encarga de determinar la posición y orientación del efector final (pinza, herramienta, sensor) a partir de los valores conocidos de las articulaciones o ejes del robot.
- Cinemática inversa: Encontrar los valores de las articulaciones necesarios para alcanzar una posición deseada en el espacio.
- Cinemática diferencial: Estudia las velocidades y aceleraciones de las articulaciones y del efecto final.



Cinemática de un brazo robot antropomorfo.

Fuente: https://commons.wikimedia.org/wiki/File:Bra%C3%A7_robot_antropom%C3%B2rfic.svg. Licencia: Creative Commons. Autor: EBatlleP

6. Nuevas formas de interacción.

De forma clásica de interacción con sistemas de inteligencia artificial han sido las típicas aplicaciones con cuadros de texto, selectores y botones entre otros, usados por personas con altos conocimientos técnicos de forma habitual.

En los últimos años, junto con la explosión del uso de IA por parte del público en general las formas de usar las IA han cambiado, haciéndolas mucho más naturales, personalizadas y adaptativas.

Las nuevas formas de interacción más destacadas:

Forma de interacción	Descripción	Ejemplos	Tecnologías asociadas
----------------------	-------------	----------	-----------------------

Lenguaje natural (voz y texto)	Comunicación con el sistema usando lenguaje humano, sin comandos técnicos.	Asistentes virtuales (Siri, Alexa, ChatGPT), chatbots de servicio al cliente.	Procesamiento de Lenguaje Natural (PLN), modelos de lenguaje, síntesis y reconocimiento de voz.
Interacción multimodal	Uso combinado de voz, gestos, imágenes o dispositivos de realidad aumentada/virtual.	Control por voz y gestos en videojuegos VR, cirugías asistidas con RA.	Visión por computadora, reconocimiento de gestos, sensores hápticos, realidad aumentada/virtual.
Interacción afectiva/emocional	Sistemas que reconocen y responden a emociones humanas.	Robots sociales, tutores virtuales que detectan frustración del estudiante.	Análisis de expresiones faciales, reconocimiento de tono de voz, IA afectiva.
Interacción adaptativa/personalizada	La IA se ajusta al contexto, preferencias y necesidades del usuario.	Sistemas de recomendación (Netflix, Spotify), educación personalizada en línea.	Aprendizaje automático, sistemas de recomendación, perfiles dinámicos de usuario.
Interacción físico-robótica	Relación con robots que actúan en el mundo físico.	Robots de compañía, robots de servicio en hoteles/hospitales.	Robótica autónoma, sensores, algoritmos de control, IA cognitiva.

Por ejemplo, en el desarrollo de software las extensiones o “plugins” en los IDE’s han adoptado la IA. Estos plugins utilizan las diferentes nuevas formas de interacción, por ejemplo:

Forma de interacción	Cómo encajan los plugins de IDE	Ejemplos
Lenguaje natural (voz/texto)	Plugins que permiten escribir código o comandos en lenguaje natural, y el sistema los traduce a código fuente.	GitHub Copilot, ChatGPT Code Plugin, CodeWhisperer.
Interacción multimodal	Integración de diagramas, autocompletado visual, depuración con gráficos, uso de UML o flujos junto al código.	PlantUML plugin, Visual Paradigm plugin.

Interacción afectiva/emocional	(Todavía emergente) Plugins que detectan patrones de estrés o errores frecuentes y sugieren pausas o mejores prácticas.	Herramientas de bienestar para desarrolladores (ej. FlowTime Tracker integrable con IDE).
Interacción adaptativa/personalizada	Plugins que aprenden del estilo de programación del usuario y ajustan las sugerencias o refactorizaciones.	SonarLint, TabNine, personalización de IntelliJ con Machine Learning.
Interacción físico-robótica	Menos común, pero algunos plugins permiten controlar hardware o robots directamente desde el IDE.	Plugins de ROS (Robot Operating System) en VSCode o Eclipse.

7. Actividades.

Estas actividades tienen la finalidad de afianzar los contenidos de la unidad, se pueden resolver casi en su totalidad leyendo los apuntes.

- 7.1. Indicar los sustitutos del Test de Turing.
- 7.2. Los datasheet son ampliamente utilizados en la inteligencia artificial. Uno de los sitios más populares es <https://www.kaggle.com>.
 Buscar datasheet, indicando el formato en que se encuentran para:
 - Visión por computador:
 - Colección de imágenes de arroz.
 - Resonancia magnética.
 - Gatos.
 - Ciencias de la computación.
 - Tendencias académicas.
 - Coches.
 - Ventas de Bmw (10-24)
 - Clasificación.
 - Datos del Titanic.
- 7.3. ¿Cuáles son las áreas fundamentales de la IA?.
- 7.4. Clasificar los siguientes problemas en: analizar grandes volúmenes de datos, identificación de patrones, toma de decisión (un problema puede clasificarse en más de un grupo).
 - Previsión de desplazamiento por carretera en Semana Santa.
 - Cálculo de compras de Amazon para gestionar el stock.
 - Evolución de una epidemia de gripe en función.
 - Análisis de ecografías en busca de anomalías.
 - Ciberataque.
 - Marketing personalizado en plataformas de streaming.
 - Prevención de averías mecánicas en función del sonido.

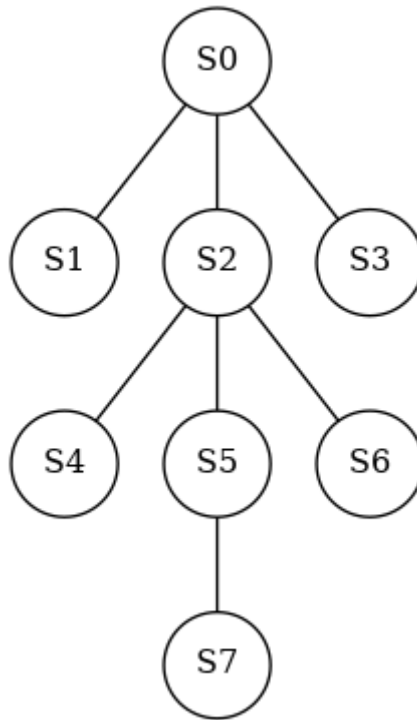
- Recomendaciones de inversión en función de las características del usuario.
 - Gestión de semáforos en una gran ciudad dependiendo del tráfico.
- 7.5. ¿Qué dos tipos de complejidades tiene un algoritmo? ¿Cuál de las dos es más determinante para su uso?
 - 7.6. Listar los principales algoritmos de búsqueda no informada. ¿Cómo se selecciona el siguiente paso a ejecutar?
 - 7.7. ¿Existen algoritmos de búsqueda especializados en juegos? ¿De qué tipo de juegos son los indicados en los apuntes?
 - 7.8. Listar los principales algoritmos de búsqueda informada. ¿Cómo se selecciona el siguiente paso en este tipo de algoritmo?
 - 7.9. Comparar las complejidades de los algoritmos informados de los no informados.
 - 7.10. La previsión de lluvia ¿Qué tipo de representación de conocimiento es?
 - 7.11. Si se almacena el conocimiento: ave—>tiene—>plumas, se trata de:
 - 7.12. ¿Qué es la inferencia? Indicar las técnicas de inferencia.
 - 7.13. Ejemplo de inducción errónea (humor), no es obligatorio (clásico de los Monty Python). <https://www.youtube.com/watch?v=Ux6fBfXOluc>
 - 7.14. Dibujar las diferentes fases por las que pasa un sistema de procesamiento de lenguaje natural.
 - 7.15. ¿Qué elementos comparten un analizador lingüístico y un compilador?
 - 7.16. Relacionar los sistemas expertos con las formas de representar el conocimiento.
 - 7.17. En los sistemas expertos ¿cuál es la función de los motores de inferencia?
 - 7.18. ¿Cuáles son los principales problemas a los que se enfrenta un robot?

8. Actividades autoevaluación.

Ejercicios básicos, se profundizará en cada de las unidades con las técnicas de IA, son simplemente una introducción.

- 8.1. Crear un programa para comparar la complejidad temporal del algoritmo de búsqueda en un vector de n elementos ordenados usando la búsqueda secuencial y la búsqueda binaria. Se puede usar cualquier lenguaje de programación.
 - Hacer una pequeña tabla(hoja de cálculo) con diferentes tamaños (10,100...) y estimar el coste al buscar en un array de 10^6 y 10^9 elementos.
 - Comprobar que se cumple que la búsqueda binaria tiene orden de complejidad $\log(n)$ y la secuencial n .
 - Crear una gráfica en que se vea como evoluciona el tiempo en función del tamaño del problema y la complejidad temporal.
 - Buscar un número que no se encuentre en el array para forzar el peor caso.

A partir del siguiente árbol:



- 8.2. Realizar un recorrido en profundidad (DFS)
- 8.3. Realizar un recorrido en anchura(BFS)
- 8.4. Representar semánticamente un equipo informático, que puede ser un ordenador sobremesa, un ordenador portátil, un móvil o un reloj digital, pudiendo usar entre relaciones entre conceptos: “es_un”, “puede_ser”, “tiene”, “se utiliza”

9. Soluciones actividades de autoevaluación.

9.1.

Python:

```
import sys
import random

def generar_array_ordenado(n: int) -> list[int]:

    return sorted(random.sample(range(1, n * 10), n))
"""
Devuelve un par posición e iteraciones.
Si no lo encuentra devuelve (-1, iteraciones).
"""
def busqueda_secuencial(array: list[int], objetivo: int) ->
tuple[int, int]:

    iteraciones = 0
    for i, valor in enumerate(array):
        iteraciones += 1
```

```

        if valor == objetivo:
            return i, iteraciones
    return -1, iteraciones
"""
Devuelve un par posición e iteraciones.
Si no lo encuentra devuelve (-1, iteraciones).
"""
def busqueda_binaria(array: list[int], objetivo: int) -> tuple[int,
int]:

    izquierda, derecha = 0, len(array) - 1
    iteraciones = 0

    while izquierda <= derecha:
        iteraciones += 1
        medio = (izquierda + derecha) // 2
        if array[medio] == objetivo:
            return medio, iteraciones
        elif array[medio] < objetivo:
            izquierda = medio + 1
        else:
            derecha = medio - 1

    return -1, iteraciones

if __name__ == "__main__":

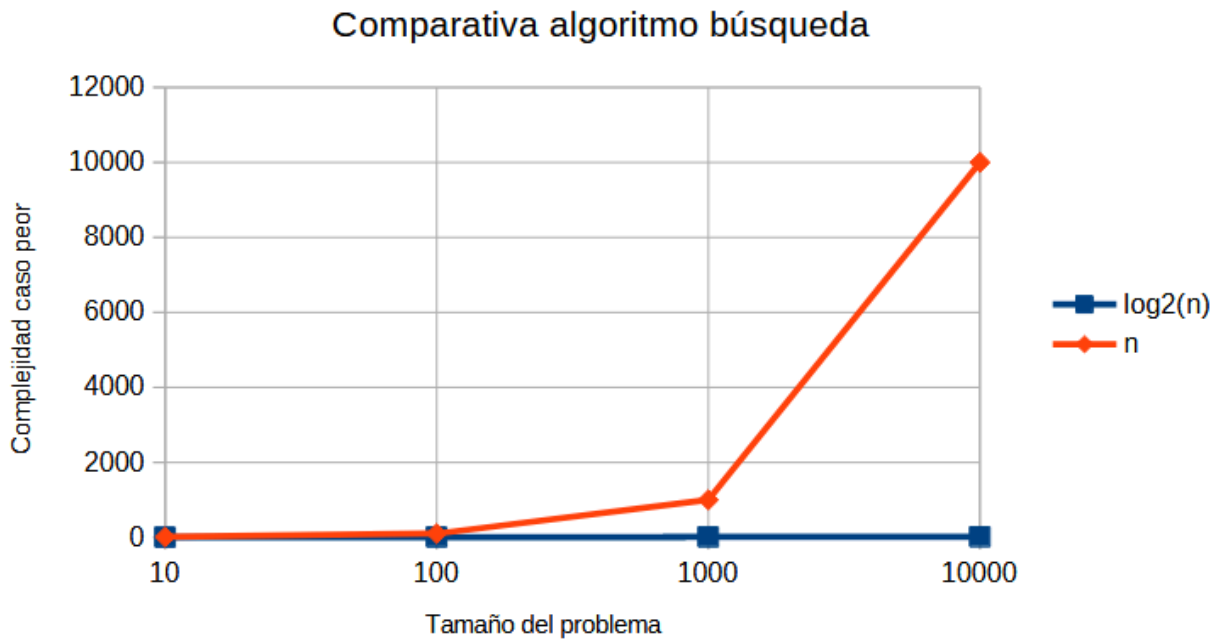
    if len(sys.argv) != 3:
        print("Uso: python3 busqueda.py <n> <objetivo>")
        sys.exit(1)
    n = int(sys.argv[1])
    objetivo = int(sys.argv[2])
    array = generar_array_ordenado(n)
    #resultados de los dos tipos de búsqueda
    indice_seq, it_seq = busqueda_secuencial(array, objetivo)
    indice_bin, it_bin = busqueda_binaria(array, objetivo)
    print(f"Busqueda binaria: {it_bin}")
    print(f"Búsqueda secuencial:{it_seq}")

```

Si se tiene tamaños de problema $n=\{10,100,1000,1000\}$ el número de iteraciones es:

			teórico binaria	teórico secuencial
n	binaria	secuencial	$\log_2(n)$	n
10	3	10	3,321928094887 36	10

100	6	100	6,643856189774 73	100
1000	10	1000	9,965784284662 09	1000
10000	13	10000	13,28771237954 95	10000



9.2.

Pila inicial: [S0]

Sacar S0, imprimir → S0

Apilar hijos de S0 en orden inverso → pila [S3, S2, S1]

Pila: [S3, S2, S1]

Sacar S1, imprimir → S1

Apilar hijos de S1 NO TIENE → pila [S3, S2]

Pila: [S3, S2]

Sacar S2, imprimir → S2

Apilar hijos de S2 en orden inverso → pila [S3, S6, S5, S4]

Pila: [S3, S6, S5, S4]

Sacar S4, imprimir → S4

S4 no tiene hijos → pila [S3, S6, S5]

Pila: [S3, S6, S5]

Sacar S5, imprimir → S5

Apilar hijos de S5 en orden inverso → pila [S3, S6, S7]

Pila: [S3, S6, S7]

Sacar S7, imprimir → S7

S7 no tiene hijos → pila [S3, S6]

Pila: [S3, S6]

Sacar S6, imprimir → S6

S6 no tiene hijos → pila [S3]

Pila: [S3]

Sacar S3, imprimir → S3

Recorrido: S0 S1 S2 S4 S5 S7 S6 S3

9.3.

Cola inicial: [S0]

Desencolar S0, imprimir → S0

Encolar hijos de S0 → cola [S1, S2, S3]

Cola: [S1, S2, S3]

Desencolar S1, imprimir → S1

S1 no tiene hijos → cola [S2, S3]

Cola: [S2, S3]

Desencolar S2, imprimir → S2

Encolar hijos de S2 → cola [S3, S4, S5, S6]

Cola [S3, S4, S5, S6]

Desencolar S3, imprimir → S3

S3 no tiene hijos → cola [S4, S5, S6]

Cola: [S4, S5, S6]

Desencolar S4, imprimir → S4

S4 no tiene hijos → cola [S5, S6]

Cola: [S5, S6]

Desencolar S5, imprimir → S5

Encolar hijos de S5 → cola [S6, S7]

Cola: [S6, S7]

Desencolar S6, imprimir → S6

S6 no tiene hijos → cola [S7]

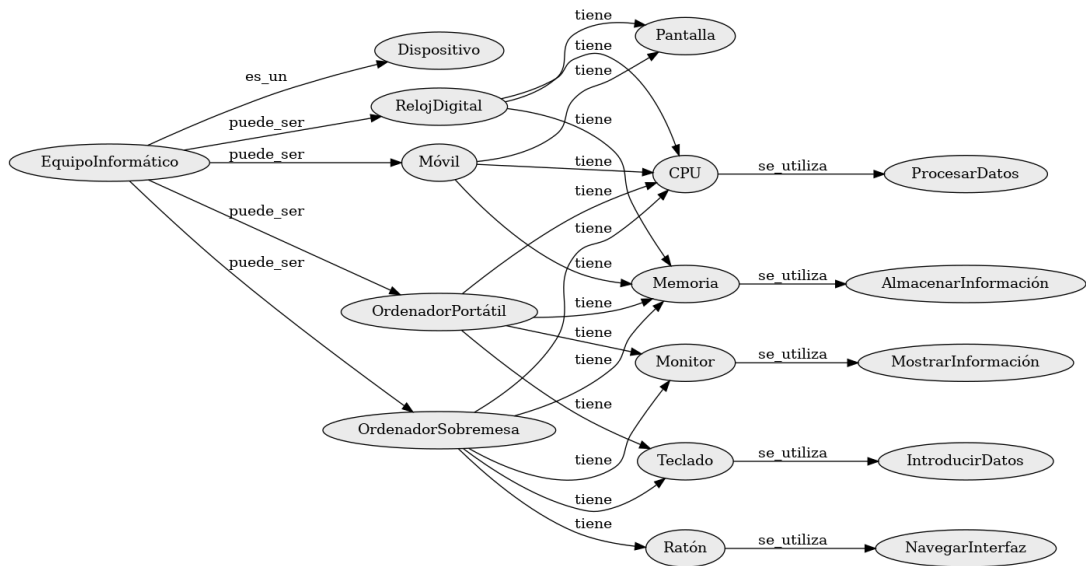
Cola: [S7]

Desencolar S7, imprimir → S7

S7 no tiene hijos → cola []

Recorrido: S0 S1 S2 S3 S4 S5 S6 S7

9.4.



Possible solution, but not unique.